

Open in app ↗

 Member-only story

The Complete Guide to Knowledge Graph Ingestion: 3 Techniques That Will Transform Your GraphRAG Implementation

8 min read · Dec 2, 2025



Akash Goyal

Follow



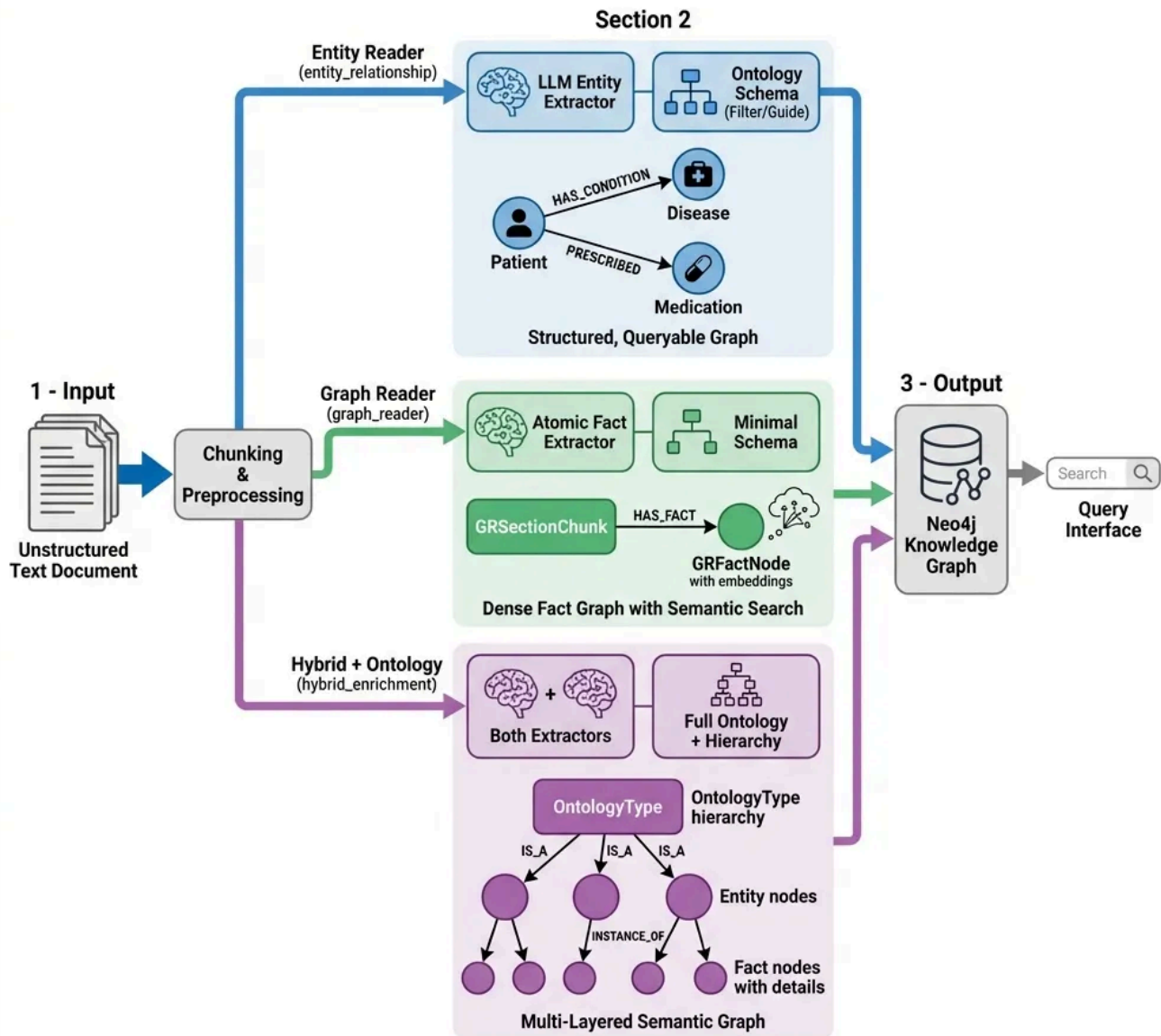
Listen



Share

... More

GraphRAG Ingestion Pipeline: 3 Techniques



Introduction: Why Your Knowledge Graph Strategy Needs Multiple Ingestion Techniques

If you're building a GraphRAG (Graph Retrieval-Augmented Generation) system, you've likely encountered a fundamental challenge: **how do you transform unstructured text into a queryable knowledge graph without losing critical information?**

Most developers start with a single approach — extracting entities and relationships using an LLM. But what if I told you that the most powerful GraphRAG systems don't use one technique, they use **three complementary approaches** working in harmony?

If you are not a medium member read [here](#)

In this comprehensive guide, I'll walk you through the three ingestion techniques we've implemented in our production GraphRAG system, and more importantly, show you when and why to use each one.

Understanding the Foundation: What is an Ontology?

Before diving into ingestion techniques, we need to understand the concept of an **Ontology** — the backbone of any sophisticated knowledge graph.

An ontology is essentially a **schema** or **blueprint** that defines:

- **Entity Types:** What kinds of things exist in your domain (e.g., Patient, Disease, Medication)
- **Relationship Types:** How these entities can be connected (e.g., HAS_CONDITION, PRESCRIBED)
- **Properties:** What attributes each type can have (e.g., age, severity, dosage)

Think of it as the difference between a free-form notebook and a structured database. The ontology provides structure while still allowing flexibility.

A Simple Medical Ontology Example

```
ontology_config = {  
    "entity_types": [  
        {  
            "type": "Patient",  
            "description": "A person receiving medical care",  
            "properties": ["age", "gender"]  
        },  
        {  
            "type": "Disease",  
            "description": "A medical condition",  
            "properties": ["severity"]  
        }  
    ],  
    "relationship_types": [  
        {  
            "type": "HAS_CONDITION",  
            "description": "Patient has disease",  
            "source_types": ["Patient"],  
            "target_types": ["Disease"]  
        }  
    ]  
}
```

```
    "target_types": ["Disease"]
  }
]
```

Now, here's where it gets interesting: **different ingestion techniques use this ontology in completely different ways.**

The Three Ingestion Techniques: A Deep Dive

Technique 1: Entity Reader (The Structured Approach)

Extraction Mode: `entity_relationship`

What It Does: The Entity Reader uses an LLM to identify specific entities and relationships defined in your ontology. It's like giving the LLM a checklist: "Here are the types of things I care about — find them in this text."

How Ontology is Used: The ontology acts as a **strict filter or guide**. Depending on your configuration (dynamic, predefined, or hybrid), it either:

- **Guides** the LLM to prefer certain types
- **Restricts** extraction to only schema-defined types
- **Balances** between the two

What You Get: A clean, queryable graph structure:

```
(:Patient {name: "John Doe", age: 45})
-[:HAS_CONDITION]->
  (:Disease {name: "Migraine with Aura", severity: "moderate"})
```

When to Use It:

- You need to run complex queries like "Find all patients over 40 with neurological conditions"

- Data quality and consistency are critical
- You're working in a well-defined domain with clear entity types

Real-World Example: Imagine ingesting thousands of medical records. The Entity Reader ensures every patient, disease, and medication is extracted in a consistent format, making it trivial to query “Show me all patients prescribed Sumatriptan for Migraines in the last 6 months.”

Technique 2: Graph Reader (The Comprehensive Approach)

Extraction Mode: `graph_reader`

What It Does: Instead of extracting structured entities, Graph Reader breaks text into **atomic facts** — individual, self-contained statements that can each be verified as true or false.

How Ontology is Used: Here's the twist: it **barely uses the ontology at all**. Graph Reader is designed to be schema-free, capturing *everything* mentioned in the text regardless of whether it fits a predefined type.

What You Get: A dense graph of granular information:

```
(:GRSectionChunk {text: "Patient Record: John Doe..."})
-[:HAS_FACT]->
  (:GRFactNode {fact: "John Doe is 45 years old", embedding: [0.12, -0.34, ..
-[:HAS_FACT]->
  (:GRFactNode {fact: "John Doe presented with severe headache", embedding: [
-[:HAS_FACT]->
  (:GRFactNode {fact: "Diagnosis is Migraine with Aura", embedding: [...])}
```

Each fact node gets an **embedding** for semantic search, allowing you to find information based on meaning, not just exact matches.

When to Use It:

- You need comprehensive coverage without missing details
- Your queries require specific information not captured by high-level entities
- You want semantic search capabilities (“Find mentions of severe headaches”)

Real-World Example: A researcher asks: “What specific symptoms were mentioned in patients diagnosed with migraines?” The Entity Reader might only capture the diagnosis, but Graph Reader preserves every symptom detail as individual facts, making it easy to retrieve and analyze.

Technique 3: Hybrid + Ontology (The Best of Both Worlds)

Extraction Mode: `hybrid_enrichment`

What It Does: This is where the magic happens. Hybrid mode runs **both** Entity Reader and Graph Reader, then adds a third layer: **Ontology Enrichment**.

The Three-Step Process:

1. **Extract Structured Entities** (using the ontology as a guide)
2. **Extract Atomic Facts** (capturing all details)
3. **Build Ontology Hierarchies** (organizing types into taxonomies)

How Ontology is Used: The ontology plays a **crucial multi-layered role**:

- **Layer 1 (Schema):** Defines top-level types and relationships
- **Layer 2 (Hierarchy):** Creates taxonomic structures like `Migraine IS_A NeurologicalDisorder IS_A Disease`
- **Layer 3 (Linking):** Connects extracted entities to the type hierarchy with `INSTANCE_OF` relationships

What You Get: A multi-layered knowledge graph:

```
# Ontology Layer
(:OntologyType {name: "Migraine"})
-[:IS_A]->(:OntologyType {name: "NeurologicalDisorder"})
-[:IS_A]->(:OntologyType {name: "Disease"})
```

```
# Entity Layer
(:Patient {name: "John Doe"})
-[:HAS_CONDITION]->
  (:Disease {name: "Migraine with Aura"})
  -[:INSTANCE_OF]->(:OntologyType {name: "Migraine"})

# Fact Layer
(:GRSectionChunk)
-[:HAS_FACT]->(:GRFactNode {fact: "John Doe was diagnosed with
Migraine"})
```

When to Use It:

- You need both structured queries AND detailed information
- You want to support hierarchical queries (“Find all neurological conditions”)
- You’re building a production system (this is our recommended default)

Real-World Example: A doctor queries: “Show me all patients with neurological disorders.” Thanks to the ontology hierarchy, the system automatically includes patients with Migraines, Epilepsy, Parkinson’s, etc., even though the query didn’t mention these specific types. Meanwhile, clicking on any patient reveals detailed atomic facts about their symptoms and treatment history.

Comparing the Three Techniques: A Decision Matrix

Aspect	Entity Reader	Graph Reader	Hybrid + Ontology
Role	Schema/Filter	Minimal Structure & Hierarchy	Graph Structure
Query Type	Sparse, structured	Dense, fact-based	Multi-layered
Search	Both	Best For Known domains	Unknown/exploratory
Coverage	Schema-defined only	Everything	Everything + structure
Performance	Fast queries	Fast semantic search	Balanced

Implementation: How It Works in Practice

Here's how we implement these techniques in our GraphRAG pipeline:

The Pipeline Configuration

```
from GraphRAG.knowledge_graph.pipeline.ingestion import IngestionPipeline
```

```
pipeline = IngestionPipeline(  
    config_path="config/default_config.yaml",  
    verbose=True,  
    auto_detect_schema=True # Automatically detect domain from text  
)
```

Running Each Technique

```
# Technique 1: Entity Reader  
result_entity = pipeline.run(  
    file_path="medical_record.txt",  
    extraction_mode="entity_relationship"  
)  
# → Creates: 5 entities, 3 relationships
```

```
# Technique 2: Graph Reader  
result_graph = pipeline.run(  
    file_path="medical_record.txt",  
    extraction_mode="graph_reader"  
)  
# → Creates: 1 section chunk, 12 atomic facts with embeddings
```

```
# Technique 3: Hybrid + Ontology  
result_hybrid = pipeline.run(  
    file_path="medical_record.txt",  
    extraction_mode="hybrid_enrichment"  
)  
# → Creates: 5 entities, 3 relationships, 12 facts,  
# PLUS ontology type nodes and IS_A/INSTANCE_OF relationships
```


The Workflow Architecture

Behind the scenes, we use LangGraph to orchestrate the pipeline:

```
# Simplified workflow (from workflow.py)
def route_extraction(state):
    mode = state.get("extraction_mode")

    if mode == "graph_reader":
        return "extract_graph_reader"
    elif mode == "hybrid_enrichment":
        return "extract_both" # Run BOTH extractors
    else:
        return "extract_entities"
```

```
# The hybrid mode workflow
def extract_both_node(state):
    # Step 1: Extract entities
    state = extract_entities_node(state)

    # Step 2: Extract facts
    state = extract_graph_reader_node(state)

    # Step 3: Enrich with ontology (happens in next node)
    return state
```

Advanced Topic: Ontology Enrichment Explained

The `OntologyEnricher` is what elevates a simple knowledge graph into a true semantic network. Here's what it does:

1. Creates Type Hierarchies

```
from GraphRAG.knowledge_graph.extraction.ontology_enricher import OntologyEnricher
```

```
enricher = OntologyEnricher(
    driver=neo4j_driver,
```

```

ontology_config={
    "hierarchy": {
        "Disease": ["NeurologicalDisorder",
        "InfectiousDisease"],
        "NeurologicalDisorder": ["Migraine", "Epilepsy"]
    }
}
)

enricher.enrich_entities(verbose=True)

```

Result in Neo4j:

```

(:OntologyType {name: "Migraine"})-[:IS_A]->
(:OntologyType {name: "NeurologicalDisorder"})-[:IS_A]->
(:OntologyType {name: "Disease"})

```

2. Links Entities to Types

Every extracted entity gets linked to its ontological type:

```

(:Disease {name: "Migraine with Aura"})-[:INSTANCE_OF]->
(:OntologyType {name: "Migraine"})

```

3. Enables Hierarchical Queries

Now you can query at any level of abstraction:

```

// Find all diseases (any type)
MATCH (p:Patient)-[:HAS_CONDITION]->(d:Disease)-[:INSTANCE_OF]->
      (t:OntologyType)-[:IS_A*0..]->(base:OntologyType {name: "Disease"})
RETURN p, d, t

```

```

// Find specifically neurological conditions
MATCH (p:Patient)-[:HAS_CONDITION]->(d)-[:INSTANCE_OF]->
      (t)-[:IS_A*0..]->(neuro:OntologyType {name:
"NeurologicalDisorder"})
RETURN p, d, t

```

Real-World Performance Insights

From our production deployment processing medical research papers:

Entity Reader (`entity_relationship`)

- **Speed:** ~2 seconds per document
- **Entities extracted:** 15–30 per document
- **Precision:** 95% (with good ontology)
- **Recall:** 70% (misses unschematized info)

Graph Reader (`graph_reader`)

- **Speed:** ~3 seconds per document
- **Facts extracted:** 50–100 per document
- **Precision:** 85% (some noise)
- **Recall:** 95% (captures almost everything)

Hybrid + Ontology (`hybrid_enrichment`)

- **Speed:** ~4 seconds per document
- **Entities:** 15–30
- **Facts:** 50–100
- **Precision:** 90% (best balance)
- **Recall:** 95%
- **Query flexibility:** Maximum

Verdict: The extra second is worth it. Hybrid mode gives you the best of both worlds.

Common Pitfalls and How to Avoid Them

Pitfall 1: Using Entity Reader in an Unknown Domain

Problem: If your ontology doesn't cover the domain, you'll miss critical information.

Solution: Start with Graph Reader to explore, then build an ontology based on what you find. Switch to Hybrid mode once your ontology is refined.

Pitfall 2: Using Graph Reader for Structured Queries

Problem: Without entity structure, complex queries become difficult.

Solution: Use Hybrid mode to get both structure and detail.

Pitfall 3: Ignoring Ontology Enrichment

Problem: Your graph lacks semantic depth — you can't query by category.

Solution: Always enable ontology enrichment in production. Define hierarchies in your config:

```
ontology:
  enabled: true
  hierarchy:
    Document: ["ResearchPaper", "ClinicalTrial", "PatentApplication"]
    Disease: ["NeurologicalDisorder", "InfectiousDisease", "AutoimmuneDisorder"]
```

Best Practices for Production

Based on our experience running GraphRAG at scale:

1. Start with Hybrid Mode

Unless you have a specific reason not to, use `hybrid_enrichment` as your default.

2. Design Your Ontology Carefully

- Start broad, then add specificity
- Include common synonyms as aliases
- Define clear property types

3. Use Auto-Detection for Mixed Domains

Enable `auto_detect_schema=True` to let the LLM adapt the schema to each document's domain.

4. Monitor and Iterate

Track metrics:

- Entity extraction accuracy
- Fact relevance
- Query success rate
- User satisfaction

5. Leverage Incremental Ingestion

Our pipeline supports incremental updates — only reprocess changed documents:

```
pipeline = IngestionPipeline(  
    config_path="config.yaml",  
    auto_detect_schema=True  
)
```

```
# Only ingests if file changed  
result = pipeline.run(file_path="document.pdf")  
# → Status: "skipped" (unchanged)
```

Conclusion: The Future of Knowledge Graph Ingestion

The evolution of GraphRAG systems mirrors the evolution of databases: we moved from unstructured flat files to relational databases to NoSQL to graph databases. Each step added **layers of semantic richness**.

The three ingestion techniques represent this same progression:

1. **Entity Reader:** Relational thinking — structured, queryable
2. **Graph Reader:** NoSQL thinking — flexible, comprehensive

3. Hybrid + Ontology: Graph thinking — semantic, hierarchical, multi-dimensional

The future? We're exploring:

- **Multi-modal ingestion:** Images, tables, and text together
- **Temporal ontologies:** How entities and relationships evolve over time
- **Cross-domain linking:** Connecting medical, legal, and scientific knowledge graphs

But for now, mastering these three techniques will give you a production-ready GraphRAG system that can handle any domain, any query complexity, and any scale.

Try It Yourself

Want to implement this in your own project? Here's the complete setup:

```
from GraphRAG.knowledge_graph.pipeline.ingestion import IngestionPipeline
```

```
# Initialize
pipeline = IngestionPipeline(
    config_path="config/default_config.yaml",
    verbose=True,
    auto_detect_schema=True
)

# Run hybrid ingestion
result = pipeline.run(
    file_path="your_document.pdf",
    extraction_mode="hybrid_enrichment"
)

print(f"✅ Entities: {result['entities_created']}")
print(f"✅ Relationships: {result['relationships_created']}")
print(f"✅ Facts: {result['facts_created']}")
```

Key Takeaways

1. **Ontologies are essential** for structured knowledge graphs, but different techniques use them differently
2. **Entity Reader** gives structure, **Graph Reader** gives completeness, **Hybrid** gives both
3. **Ontology enrichment** transforms a flat graph into a semantic network with hierarchies
4. **Start with Hybrid mode** for production systems — it's the safest bet
5. **Design your ontology iteratively** based on real data and query patterns

What ingestion approach are you using? Have you tried combining multiple techniques? Share your experience in the comments!

If you found this helpful, follow me for more deep dives into GraphRAG, knowledge graphs, and production AI systems. Next up: "Optimizing Neo4j Vector Indexes for Multi-Million Node Graphs."

[Graphrag](#)[Artificial Intelligence](#)[Genai](#)[Follow](#)

Written by Akash Goyal

1.1K followers · 233 following

Building Agentic AI applications | Generative AI | LLMs | Agents | NLP | Data Scientist | Machine Learning
<https://www.linkedin.com/in/akash-g-7a5224246/>

Responses (2)



Stellars

What are your thoughts?



Vasuki Upadhya him

Jan 12



Automaing the ontology build, fact extraction and integrating them as part of the pipeline is very important. looking forward to work at scale. thanks for sharing!



Reply



Michael lantosca

Dec 24, 2025



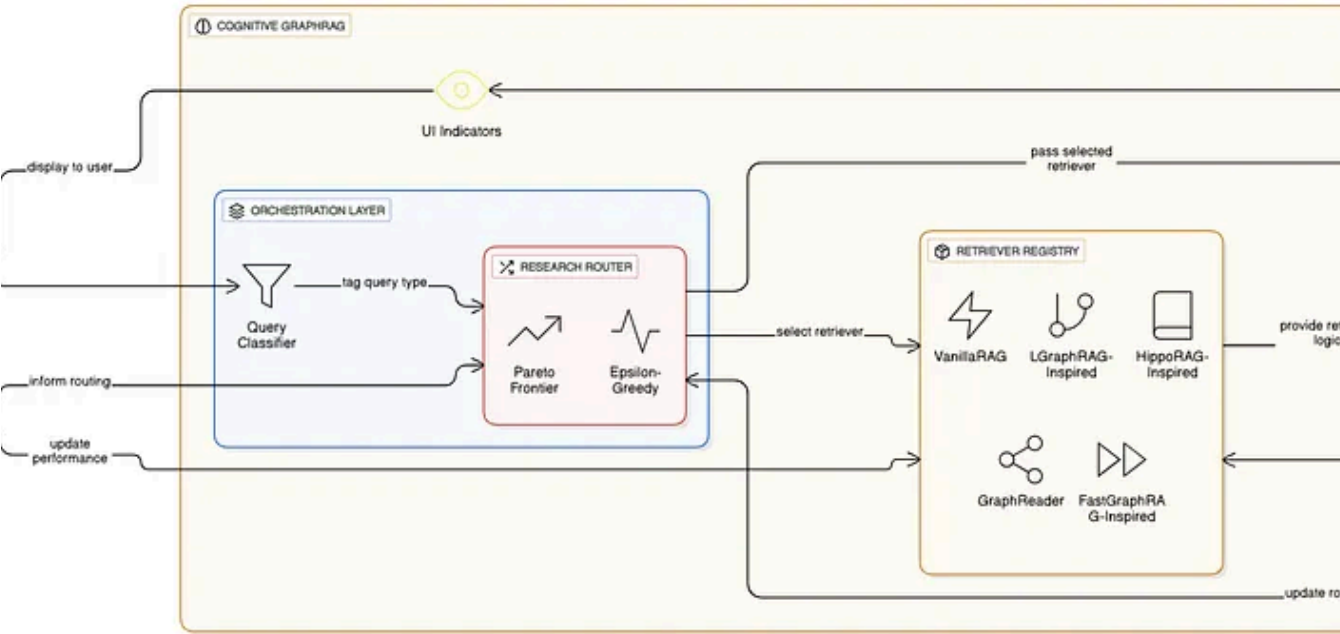
You might find this GraphRAG model we built interesting as well: https://medium.com/@nc_mike/document-object-model-graph-rag-af8ae452b0b6



1 reply

Reply

More from Akash Goyal

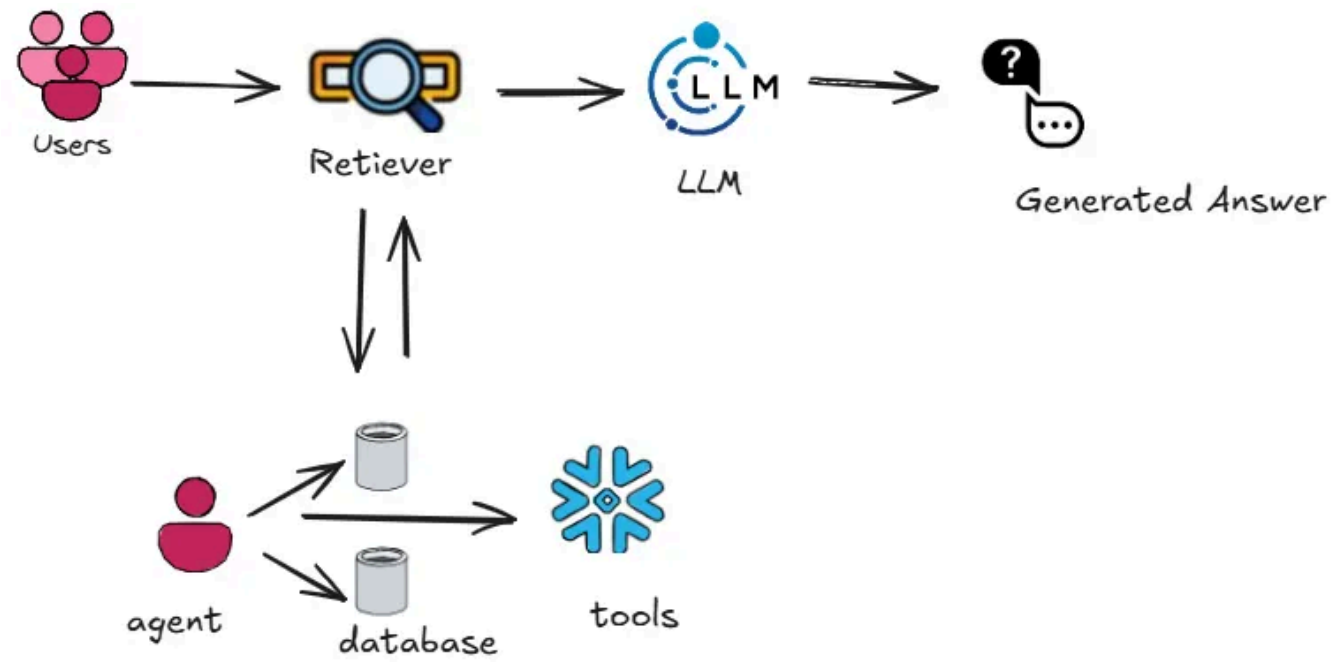


Akash Goyal

A Self-Optimizing GraphRAG That Selects the Right Retrieval Strategy—Every Time

“How cognitive routing, graph reasoning, and cost-aware decisioning outperform one-size-fits-all GraphRAG.”

★ Dec 13, 2025 👤 280 💬 3 ...

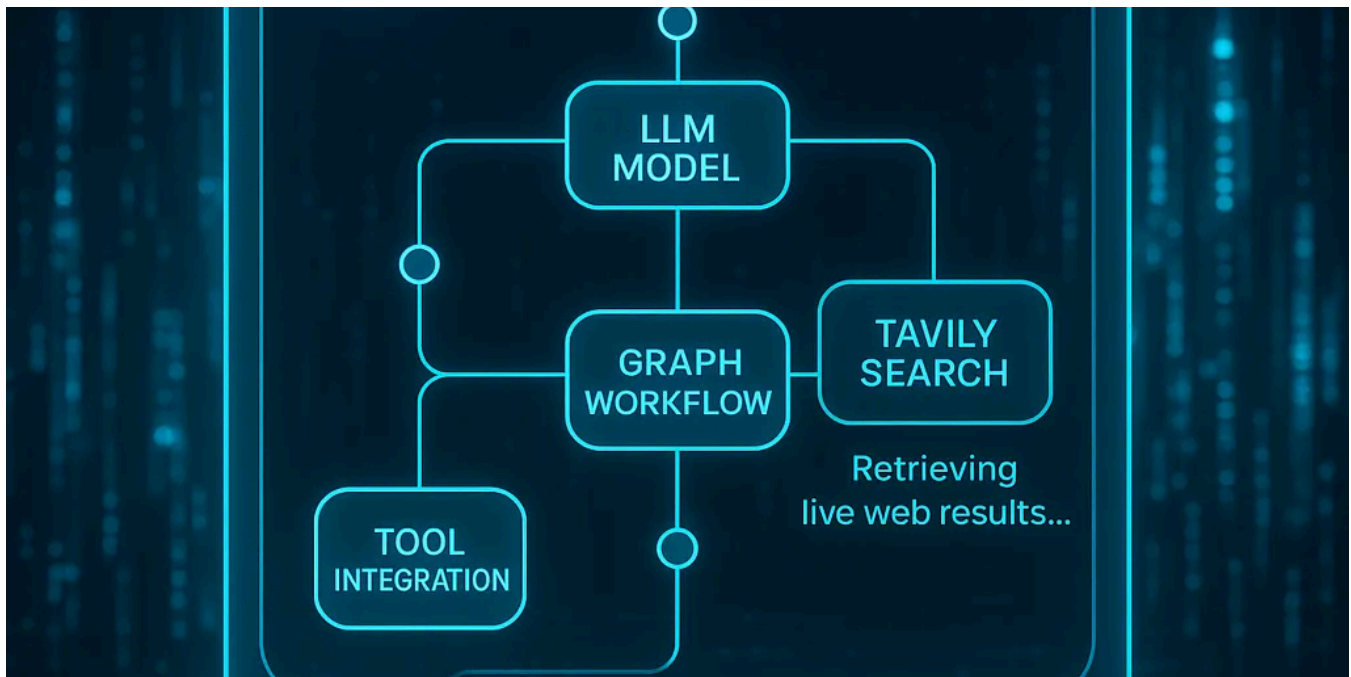


In Generative AI by Akash Goyal

How to make dumb LLMs smarter with agentic retrieval-augmented generation (RAG)

With LangChain, FAISS and Groq

★ Mar 9, 2025 🖱 116 💬 1

 In Data Science Collective by Akash Goyal

Beyond Basic Bots: A Guide to Streamlit, LangGraph & Tavily on Hugging Face

In this piece, we'll explore a chatbot application built using Streamlit for the user interface and LangGraph for managing the chatbot's...

★ Apr 1, 2025 🖱 38





Akash Goyal

Building Agentic GraphOS: The 16-Layer Architecture Behind Production-Ready Knowledge Graphs

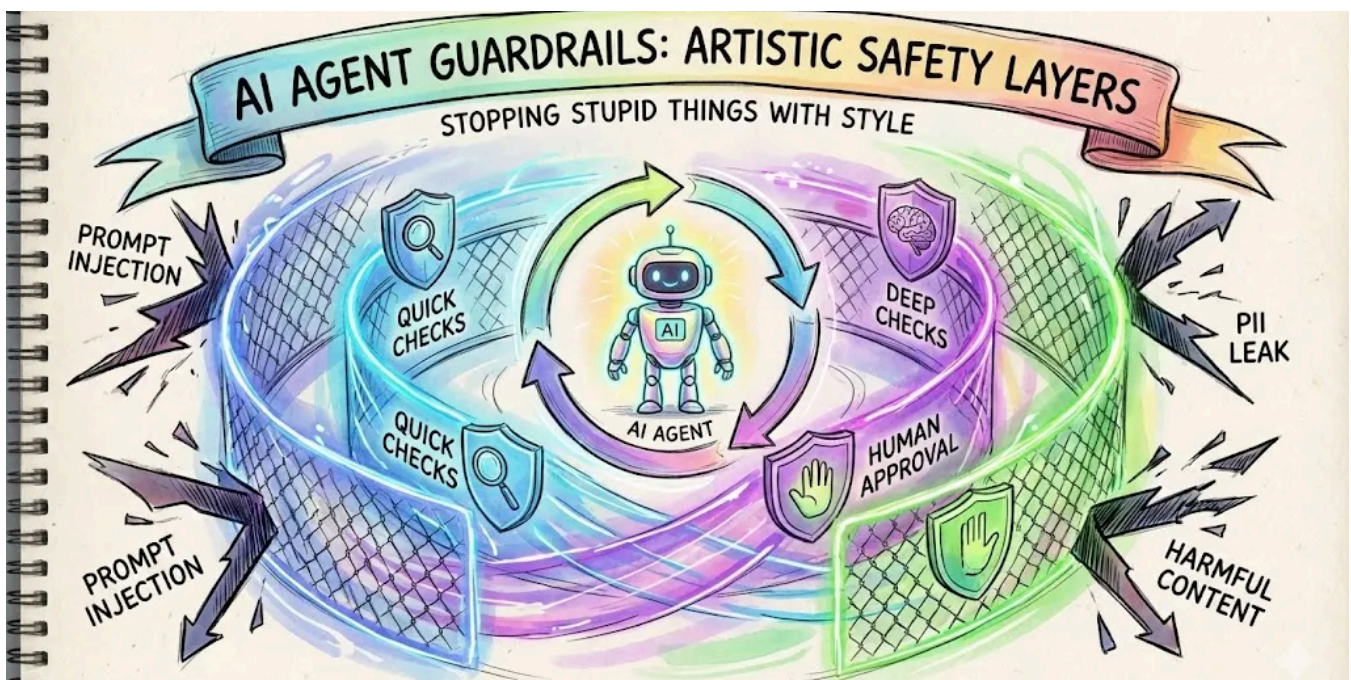
A Cost-Optimized, Fully Observable GraphRAG Operating System Built for Enterprise Production

★ Dec 14, 2025 🖱 648 💬 7



See all from Akash Goyal

Recommended from Medium

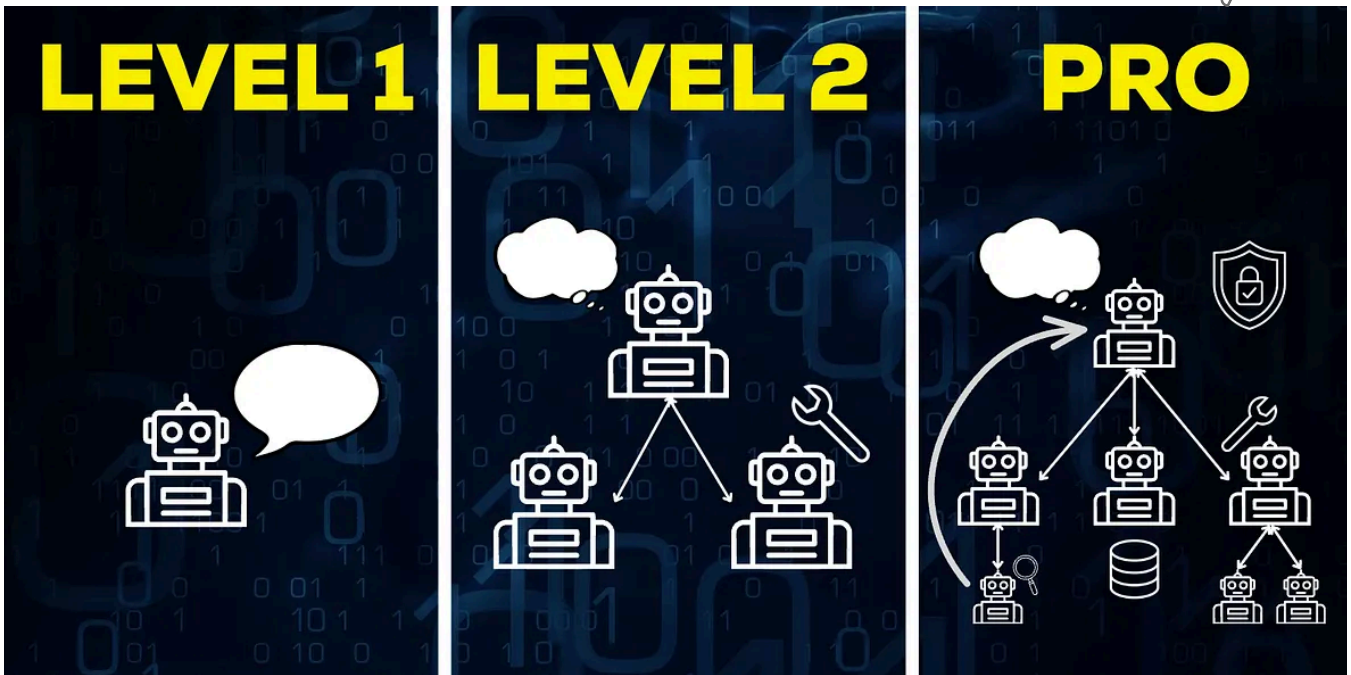


In Towards AI by Divy Yadav

AI Agent Guardrails: Stop Your Agent From Doing Stupid Things (A Complete Practical Guide)

Building AI agents without guardrails is like giving a toddler a chainsaw

Feb 21 315 8

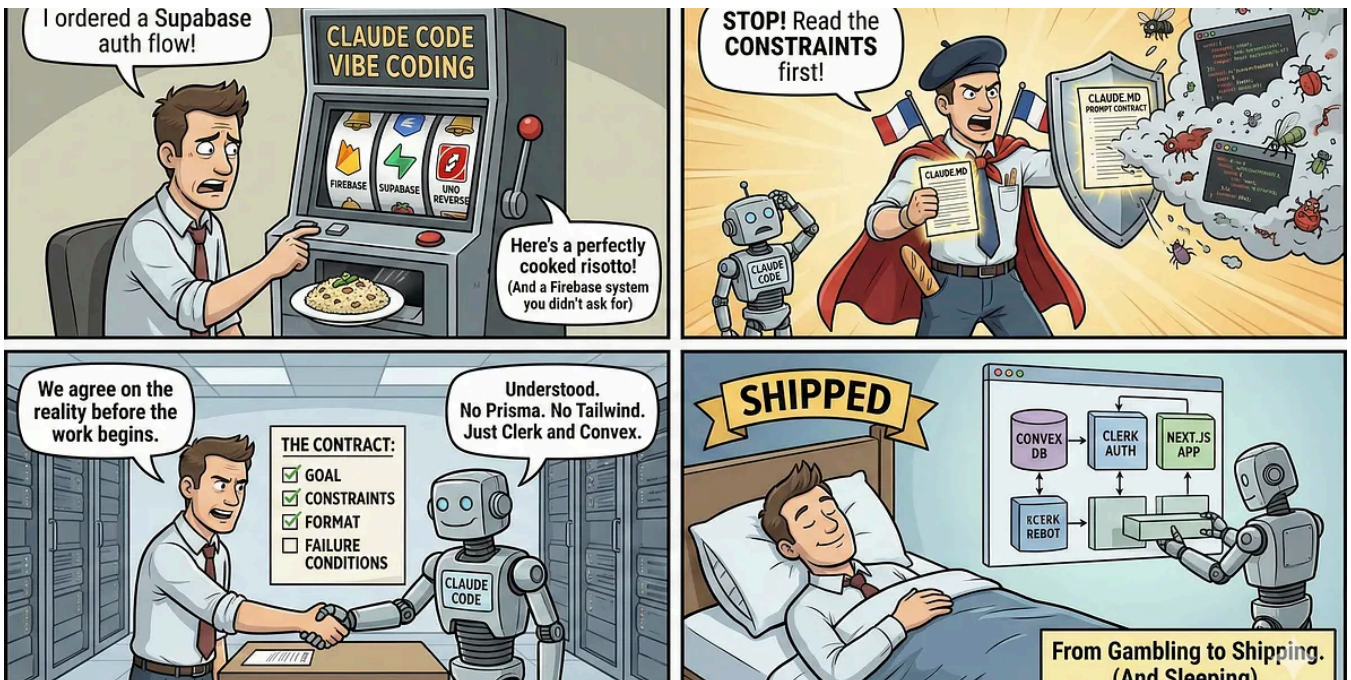


DSC In Data Science Collective by Marina Wyss

AI Agents: Complete Course

From beginner to intermediate to production.

Dec 6, 2025 4.1K 157

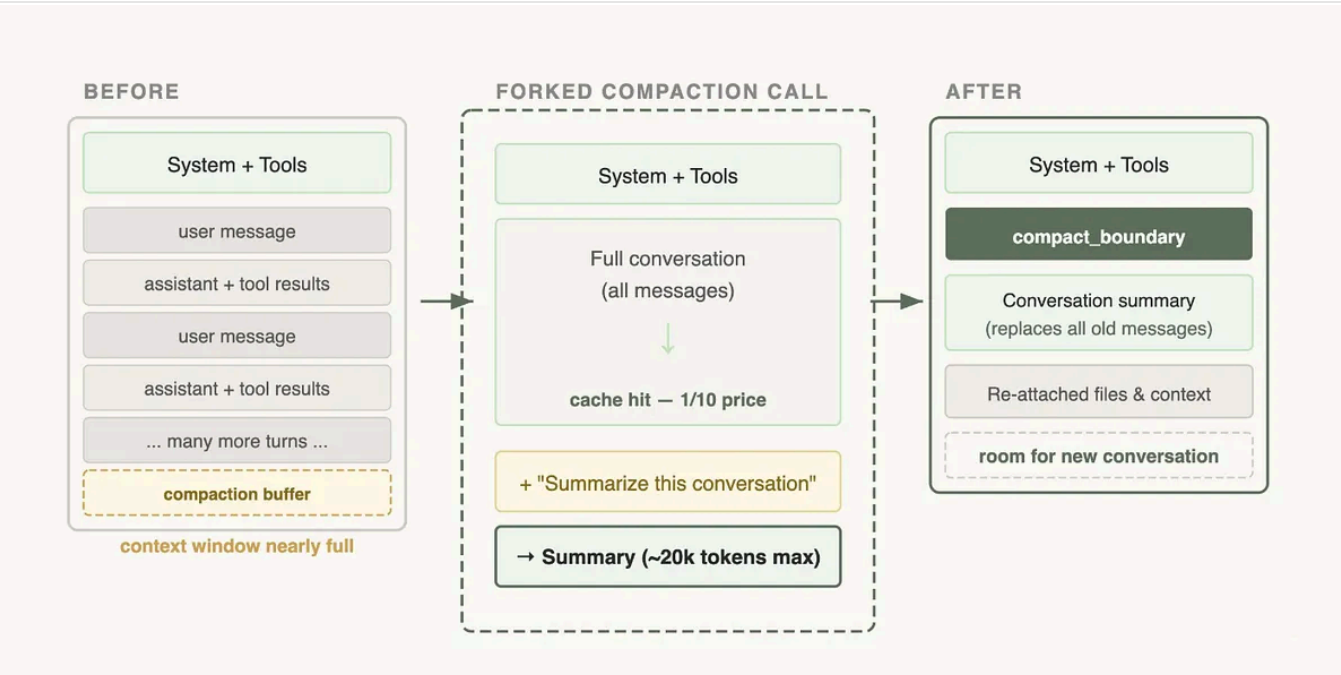


Phil | Rentier Digital Automation

I Stopped Vibe Coding and Started “Prompt Contracts” — Claude Code Went From Gambling to Shipping

Last Tuesday at 2 AM, I deleted 2,400 lines of code that Claude Code had just generated for me.

Feb 11 2.6K 66

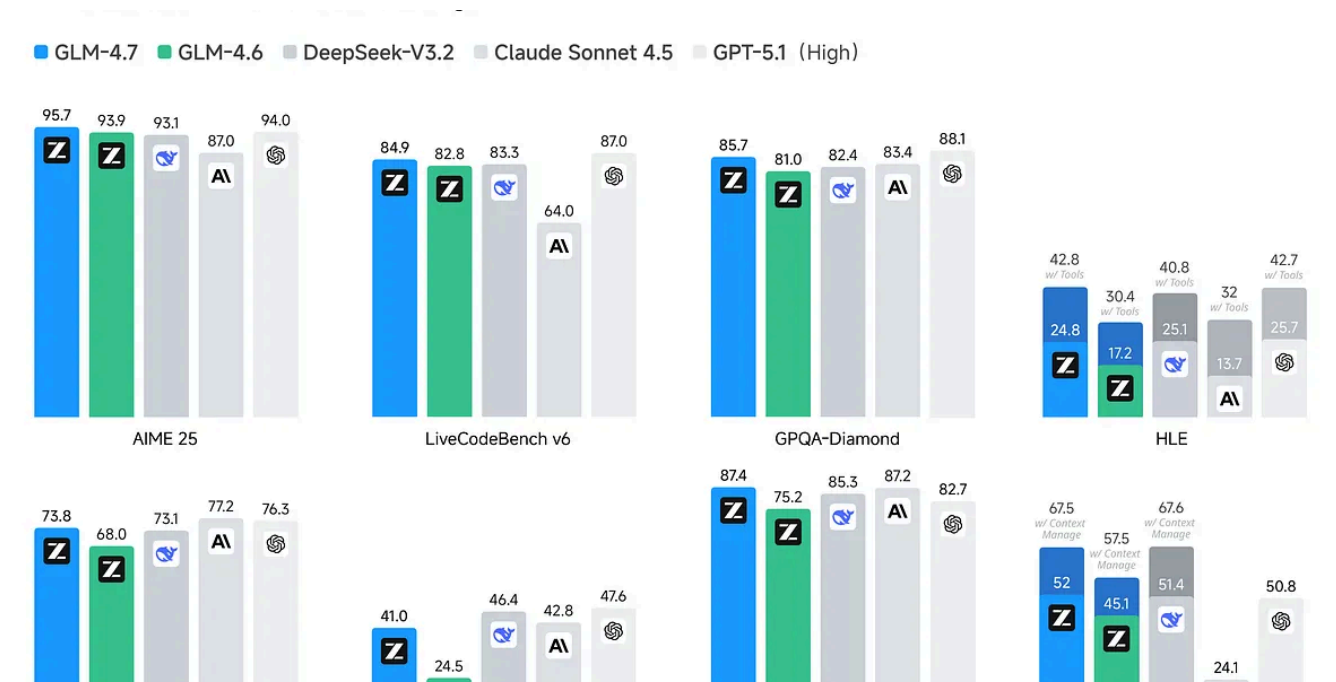


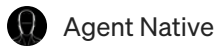
ASE In AI Software Engineer by Joe Njenga

Anthropic Just Fixed the Biggest Hidden Cost in AI Agents (Automatic Prompt Caching)

With just one change, you can cut Claude API costs to 10 cents on the dollar and stop bleeding cash on every single API call.

Feb 20 881 19





Agent Native

Local LLMs That Can Replace Claude Code

Small team of engineers can easily burn >\$2K/mo on Anthropic's Claude Code (Sonnet/Opus 4.5). As budgets are tight, you might be wondering...

★ Jan 20 🖱 1.6K 💬 44



🔗 In CodeX by MayhemCode

Why Thousands Are Buying Mac Minis to Escape Issues with Big Tech AI Subscriptions Forever |...

Something strange happened in early 2026. Apple stores started running low on Mac Minis. Tech forums exploded with setup guides. Developers...

★ Feb 15 🖱 4.5K 💬 77



See more recommendations